

Prometheus & Grafana Monitoring

1. Introduction to Monitoring

Monitoring continuously collects metrics from applications and infrastructure so that issues can be detected before they affect users. A modern monitoring stack provides visibility into CPU, memory, disk, network, applications, databases and services.

2. Prometheus Overview

Prometheus is an open-source monitoring and alerting toolkit that stores metrics as time-series data and queries them using PromQL.

- Pull-based metric collection
- Time-series database
- PromQL
- Alerting
- Service discovery
- Kubernetes support

3. Prometheus UI vs Grafana

Prometheus UI

Prometheus provides a built-in web UI mainly used for: - Checking whether metrics are being scraped Running PromQL queries - Debugging issues

Note: Prometheus UI is not ideal for dashboards.

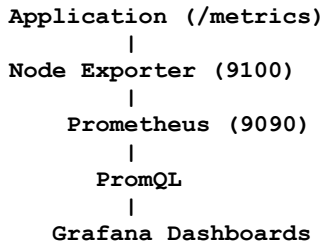
Grafana

Grafana is a visualization tool used to create:

- Interactive dashboards
- Graphs, gauges, and tables
- Alerts Grafana queries Prometheus and displays the data in a visual format.

Prometheus	Grafana
Scrapes metrics	Visualizes metrics
Runs PromQL	Builds dashboards
Checks targets	Creates alerts
Debugging	Executive dashboards

4. Architecture



5. Static Web Application

Create a Node.js application exposing /metrics using express and prom-client.

➔ Install Node.js

```

sudo apt update
sudo apt install -y nodejs npm

```

➔ Create Project Directory

```

mkdir static-web-monitoring
cd static-web-monitoring
npm init -y

```

➔ Install Dependencies

```

npm install express prom-client\

```

Create Static HTML Page

Create a folder and file:

- ➔ `mkdir public`
- ➔ `nano public/index.html`

```

<!DOCTYPE html>
<html>
<head>
<title>Simple Static App</title>
</head>
<body>
<h1>Welcome to My Static Web App</h1>
<p>Monitoring with Prometheus and Grafana</p>
</body>
</html>

```

Create Application Code:

➔ Create app.js :

```

const express = require('express');
const client = require('prom-client');
const app = express();

```

```
const port = 3000;

const register = new client.Registry();
client.collectDefaultMetrics({ register });
const httpRequestsTotal = new client.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
});
register.registerMetric(httpRequestsTotal);
app.use(express.static('public'));
app.use((req, res, next) => {
  httpRequestsTotal.inc();
  next();
});
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', register.contentType);
  res.end(await register.metrics());
});
app.listen(port, () => {
  console.log(`Application running on http://localhost:${port}`);
});
```

Run the Application:

node app.js

Access: -

Web App: <http://localhost:3000>

Metrics: <http://localhost:3000/metrics>

6. Installing Prometheus

```
cd /opt
wget
https://github.com/prometheus/prometheus/releases/download/v2.49.1/prometheus-2.49.1.linux-amd64.tar.gz
tar -xvf prometheus-2.49.1.linux-amd64.tar.gz
mv prometheus-2.49.1.linux-amd64 prometheus
```

prometheus.yml

```
global:
  scrape_interval: 15s

scrape_configs:
- job_name: prometheus
  static_configs:
    - targets: ["localhost:9090"]

- job_name: static-web-app
  static_configs:
    - targets: ["localhost:3000"]

- job_name: node-exporter
  static_configs:
    - targets: ["localhost:9100"]
```

7. Prometheus systemd Service

Create the service file:

```
sudo nano /etc/systemd/system/prometheus.service
```

Paste:

```
[Unit]

Description=Prometheus Monitoring

Wants=network-online.target

After=network-online.target

[Service]

Type=simple

User=root

Group=root

WorkingDirectory=/opt/prometheus

ExecStart=/opt/prometheus/prometheus --
config.file=/opt/prometheus/prometheus.yml

Restart=always

RestartSec=5

[Install]

WantedBy=multi-user.target
```

Reload system:

```
sudo systemctl daemon-reload
```

Enable service:

```
sudo systemctl enable Prometheus
```

Start service:

```
sudo systemctl start Prometheus
```

Check status:

```
sudo systemctl status Prometheus
```

Open:

<http://localhost:9090>

8. Installing Node Exporter

```
cd /opt

wget
https://github.com/prometheus/node\_exporter/releases/download/v1.7.0/node\_exporter-1.7.0.linux-amd64.tar.gz

tar -xvf node_exporter-1.7.0.linux-amd64.tar.gz

mv node_exporter-1.7.0.linux-amd64 node_exporter
```

9. Node Exporter systemd Service

Create the service file:

```
sudo nano /etc/systemd/system/node_exporter.service
```

Paste:

```
[Unit]
Description=Prometheus Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=root
Group=root
WorkingDirectory=/opt/node_exporter
ExecStart=/opt/node_exporter/node_exporter
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Reload system:

```
sudo systemctl daemon-reload
```

Enable service:

```
sudo systemctl enable node_exporter
```

Start service:

```
sudo systemctl start node_exporter
```

Check status:

```
sudo systemctl status node_exporter
```

Open:

```
http://localhost:9100/metrics
```

10. Install and Configure Grafana

Update the system

```
sudo apt update  
  
sudo apt install -y software-properties-common wget apt-transport-https
```

Import the Grafana GPG key

```
sudo mkdir -p /etc/apt/keyrings  
  
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee  
/etc/apt/keyrings/grafana.gpg > /dev/null
```

Add the Grafana repository

```
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com  
stable main" | sudo tee /etc/apt/sources.list.d/grafana.list
```

Update the package list

```
sudo apt update
```

Install Grafana

```
sudo apt update  
  
sudo apt install -y grafana
```

Start Grafana

```
sudo systemctl start grafana-server  
  
sudo systemctl enable grafana-server  
  
sudo systemctl status grafana-server
```

Access Grafana: <http://localhost:3000>

Default Login: - Username: admin - Password: admin

11. Configure Prometheus Data Source

1. Grafana → Connections → Data Sources
2. Add Data Source → Prometheus
3. URL: <http://localhost:9090>
4. Click Save & Test

12. PromQL Query Library

Metric	PromQL
CPU Usage	$100 - (\text{avg by(instance)}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}])) * 100$
Memory Usage %	$100 * (1 - (\text{node_memory_MemAvailable_bytes} / \text{node_memory_MemTotal_bytes}))$
Disk Usage %	$100 * (1 - (\text{node_filesystem_avail_bytes} / \text{node_filesystem_size_bytes}))$
Disk Read	$\text{rate}(\text{node_disk_read_bytes_total}[5\text{m}])$
Disk Write	$\text{rate}(\text{node_disk_written_bytes_total}[5\text{m}])$
Network RX	$\text{rate}(\text{node_network_receive_bytes_total}[5\text{m}])$
Network TX	$\text{rate}(\text{node_network_transmit_bytes_total}[5\text{m}])$
HTTP Requests	$\text{rate}(\text{http_requests_total}[1\text{m}])$
Uptime	$(\text{node_time_seconds} - \text{node_boot_time_seconds}) / 3600$
Target Health	up

13. Recommended Grafana Dashboard

- CPU
- Memory
- Disk
- Network
- Load Average
- Uptime
- HTTP Requests
- Target Health

14. Troubleshooting

- Check service status using systemctl status.
- Verify targets under <http://localhost:9090/targets>.
- Review logs using journalctl.
- Verify firewall ports 9090 and 9100.
- Validate prometheus.yml syntax.

15. Best Practices

- Run services using systemd.
- Enable Alertmanager.
- Use recording rules.
- Backup Prometheus data.
- Secure endpoints with TLS.
- Monitor exporters themselves.

16. Conclusion

This guide provides a production-style monitoring implementation using Prometheus, Node Exporter and Grafana suitable for labs, enterprise environments and interview preparation.